

Graphics and Music Challenges for Classic-Style Computer Applications

Peter Occil

This version of the document is dated 2026-10-06.

Peter Occil

The following are challenges I make to the computer community, relating to:

- **Graphics for classic-style game development.**
- **MIDI music synthesis for classic-style games and apps.**
- **Tileable wallpapers with limited colors and resolution¹.**
- **Other challenges and projects.**

All may interest 1990s computer users.

1 Contents

- **Contents**
- **Graphics Challenge for Classic-Style Games**
 - **The Specification**
 - **Classic Graphics in Scope**
 - **Optional Limits**
 - **Notes on Specification**
 - * **Screen resolutions**
 - * **Frame rate**
 - * **3-D graphics**
 - * **Graphics in video arcade games**
 - * **Screen image effects (filters)**
 - * **Sounds**
 - * **Memory**
 - **Seeking Comments**
 - **Further Reading**
- **Building a Public-Domain music synthesis library and instrument banks**
- **Other Challenges and Projects**
 - **Classic desktop wallpaper**
 - **Button and border styles for classic interfaces**
 - **Sound bank development guide**
 - **Guide for creating 3-D models in the pre-2000 style**
 - **Book-form tutorial on pre-2000 computer graphics programming**
- **Acknowledgments**
- **License**
- **Notes**

¹<https://peteroupc.github.io/classic-wallpaper>

2 Graphics Challenge for Classic-Style Games

An interesting challenge for game developers, relating to designing games with classic graphics that run on an exceptional variety of modern and recent computers.

In this document, *classic graphics* or *pre-2000 graphics* generally means two- or three-dimensional graphics achieved by video games up to some year earlier than 2000, before the advent of programmable “shaders”. For details, see “**Classic Graphics in Scope**”, later. (To summarize: In general, a game screen of 640×480 or smaller, up to 12,800 3-D polygons at a time, and tile- or sprite-based 2-D visuals are involved.)

This challenge is intended to encourage the making of—

- modern video games that simulate pre-2000 graphics and run with very low resource requirements (say, 64 million bytes of memory or less) and even on very low-end computers (say, those that date from 2010 or earlier or support Windows 7, Windows XP, or even older), and
- graphics engines (especially free and open-source ones) devoted to pre-2000 visuals and meant for developing such modern video games.²

Most desktop and laptop computers from 2010 on, and most smartphones from 2016 on, can draw even high-quality pre-2000 graphics using only software — without relying on specialized video cards — at 640×480 pixels or smaller, screen resolutions typically targeted by video games in the 1990s and earlier.³

The challenge sets an *upper bound* on the kind of computer graphics that are of interest. Further **constraints to graphics computation** (such as memory, resource, color, resolution, or triangle limits) are highly encouraged. It is also encouraged to establish a lean programming interface for this specification; for details, see “**Lean Programming Interfaces for Classic Graphics**”⁴.

2.1 The Specification

Limit 3-D graphics to the following:⁵

1. The maximum number of primitives that can be shown at a time is equal to screen width times screen height divided by 24.⁶

²The following are examples of a graphics library that follows the spirit, even if not the letter, of this specification: *Tilengine*, *kit*, *DOS-like*, *raylib*’s *rlsw* software renderer, *quadplay*. Michal Strehovský published an **interesting technique to create small game applications**, and so did Jani Peltonen. I offer a **template for Win32 applications** that supports 8- and 24-bit-per-pixel game images and paints finished game images using *StretchDIBits*. By contrast, even though today’s Unity and Unreal game engines can allow the making of games with “classic graphics”, they are far from lightweight and their use is not within the spirit of this challenge. <https://github.com/megamarc/Tilengine> <https://github.com/rxi/kit> <https://github.com/mattiasgustavsson/dos-like> <https://github.com/raysan5/raylib> <https://github.com/morgan3d/quadplay> <https://migeel.sk/blog/2024/01/02/building-a-self-contained-game-in-csharp-under-2-kilobytes/> <https://www.codeslow.com/2019/12/tiny-windows-executable-in-rust.html> <https://gist.github.com/peteroupc/f1a5d8e45e27123b86b284271cfd802b>

³A computer has adequate performance for pre-2000 graphics if it achieves a score of—(a) 3108 or more 3D marks on the 3DMark2000 benchmark (640×480) when run without hardware acceleration, or (b) 195 or greater on the 3DMark2000 CPU speed test. Both figures correspond to the running of two visually demanding 3-D demos, at three levels of detail each, at 60 frames per second (adjusted downward as needed if a demo’s detail level averages more than 12,800 triangles per frame; see the section “Test Descriptions” in the 3DMark2000 help).

⁴<https://peteroupc.github.io/graphicsapi.html>

⁵One editor specialized for creating classic 3-D models is the open-source tool *Blockbench*. For classic 3-D scenes, there is the open-source tool *Trenchbroom*. <https://www.blockbench.net/> <https://trenchbroom.github.io/>

⁶There are many **definitions of triangles** per frame. One definition is the triangles processed each frame after their vertices are transformed but before they are clipped (corresponding to the `GL_PRIMITIVES_GENERATED` query in **OpenGL 3.1**, section 2.16). Another definition (often called *visible triangles*) is the number of triangles under the first definition that are neither back-facing nor totally outside the view volume. Clipping these visible triangles to the view volume (which can sometimes produce more triangles) leads to a third definition: the triangles inside the volume that result after this clipping. A similar situation occurs with “primitives per frame” rather than “triangles per frame”. For purposes of this specification, an overestimate of primitives per frame (such as in the first definition) is better than an underestimate. The maximum number of primitives per frame in this specification was determined based on a target rate of 60 frames per second. <https://developer.arm.com/community/arm-community-blogs/b/mobile-graphics-and-gaming-blog/posts/triangles-per-second-performance-metric-or-chocolate-teapot> <https://registry.khronos.org/OpenGL/specs/gl/glspec31.pdf>

- A *primitive* is a triangle or a line segment. Alternatively, a primitive is a triangle, a line segment, or a convex quadrilateral.
 - Each vertex of the primitive points to a vertex from the vertex list described later.
 - Each primitive can be translucent.
2. The maximum number of vertices that can be displayed at a time (or rendered per second) is 3 times the maximum number of primitives.
 - A *vertex* has at least these properties: a three-dimensional position (x-, y-, and z-coordinates) and a red-green-blue *vertex color*. A vertex may also have one or more pairs of *texture coordinates* (u- and v-coordinates) depending on whether the vertex belongs to a texture-mapped or texture-blended primitive.⁷
 3. Each *texture* (an image that is applied to the surface of 3-D objects)—
 - is in a 16-bit-per-pixel format, where each pixel has a vertex color format given earlier, or
 - is in a 1-, 2-, 4-, or 8-bit-per-pixel format and has a table of colors with that color format.
 4. The width and height of each texture are each powers of 2.
 5. Image files used by the game should not store “pre-pixelated” textures. A “pre-pixelated” image results when an image is enlarged in advance with point filtering (also called nearest-neighbor filtering), with the result that some or all of the resulting image’s rows and columns are repeated.
 6. For 3-D graphics, Z buffering (depth buffering), flat shading, Gouraud shading, per-vertex specular highlighting, per-vertex depth-based fog, line drawing (by approximating the line at integer coordinates), two-texture blending, MIP mapping, source alpha blending, and destination alpha blending are supported.⁸ Bilinear filtering, edge antialiasing (smoothing)⁹, and perspective correction (outside the vertices)¹⁰ are optional.
 7. The following color formats only are supported for vertex colors:
 - The red, green, and blue components occupy up to 5 bits each. No transparent or translucent pixels.
 - The red and blue components occupy 5 bits each and the green component occupies 6 bits. No transparent or translucent pixels.
 - The red, green, and blue components occupy up to 5 bits each. A 1-bit alpha component marks transparent pixels. No translucent pixels.
 - For 3-D rendering only: The red, green, and blue components occupy 4 bits each, and a 4-bit alpha component marks transparent and translucent pixels.¹¹

Limit 2-D graphics to the following:¹²

⁷For example, see “Optimizing OpenGL drivers for Quake3” by John Carmack, referenced in .plan, April 30, 1999. But primitives with two or more textures are a feature hardly seen before the late 1990s. See that document as well as Abrash (1997), chapters 69-70.

⁸*Quake* (1996) also employed *subdivision rasterization* for drawing small and relatively distant triangles whose vertices are rounded to integers, an algorithm likewise in scope here (Abrash (1997), chapter 69).

⁹Antialiasing “**didn’t appear in home console graphics architectures** until the debut of the [Nintendo 64] in late 1996”. McCornack et al. (1995) considers antialiasing among the features “unlikely” to be needed in game programming. <https://imagequalitymatters.blogspot.com/2011/01/retro-tech-analysis-virtua-racing-md-vs.html>

¹⁰Perspective correction accounts for distance from the viewer: closer objects appear larger. The lack of perspective correction (as in what is called *affine texture mapping*) contributed to the characteristic distortion and instability of 3-D graphics in many video games for the original PlayStation; so did the fact that triangles were drawn without smooth edges and with vertices placed at integer coordinates on the game screen.

¹¹Translucent pixels enable *alpha blending* techniques (the mixing of one image with another), also known as *compositing*. But alpha blending was “relatively new to PC games” at the time of *Quake*’s launch in 1996, according to Abrash (1997), and is practically not discussed at all in McCornack et al. (1995). Only images with opaque and/or transparent pixels tended to be supported in early-1990s video games. <https://ciechanow.ski/alpha-compositing/>

¹²It is being considered whether to replace these 2-D limits with one of the following alternatives: 1. Instead of tiles, sprites, and layers, the game uses a *frame buffer* (array of color samples, called pixels, in computer memory) with no more than 8 bits

1. Layers:

1. Up to four *2-D layers* can be displayed at a time. Each 2-D layer is a rectangular array of references to *tiles* (see later), and can also be called a *tile map*.¹³
2. If 3-D graphics are being displayed, one of the 2-D layers is replaced with a *3-D layer*, which is an image on which the 3-D graphics are drawn. Which 2-D layer is replaced this way can vary over time.
3. The 2-D layers may contain transparent pixels. The 3-D layer may contain transparent and translucent (semitransparent) pixels.¹⁴
4. The upper-left corner of a 2-D layer can be placed anywhere relative to the game screen and even outside it. The 3-D layer's upper-left corner and dimensions are the same as the game screen image's.

2. Tiles. A *tile* is a small rectangular array of pixels.

1. Tiles share width in pixels, height in pixels, and number of bits per pixel.
2. If tiles have fewer than 8 bits per pixel, each tile uses one of 16 color tables. If tiles have 8 bits per pixel, they use the same 256-color table.
3. Each color in each color table used by tiles is of the vertex color format given earlier.

3. Sprites. A *sprite* is a rectangular array of either tiles or pixels.

1. Each sprite made of pixels (rather than tiles) has a pixel format allowed for *3-D textures*, given earlier.
2. Each sprite can be drawn above or below any of the 2-D or 3-D layers.
3. The upper-left corner of a sprite can be placed anywhere relative to the game screen and even outside it.

4. Image files used by the game should not store “pre-pixelated” sprites or tiles.

5. The drawing of sprites and tiles with transparency (that is, the drawing of some or all of their pixels) is supported.¹⁵ Sprites and tiles with translucent (semitransparent) pixels are not supported, and drawing them with translucency is also not supported.

Other requirements:

- **Rendering in software:** The game includes a mode in which the graphics are *rendered in software*. This means that their rendering does not rely on a video card or graphics accelerator, whether directly or indirectly (such as through GDI, OpenGL, or Direct3D), with the sole exception of sending a finished game screen image to the player's display (such as through GDI's **StretchDIBits** or copying to video memory). (Implementing a subset of **OpenGL ES**¹⁶ 1.1 or **OpenGL**¹⁷ 1.1 in software without relying on a video card or graphics accelerator is allowed.) The game can optionally support hardware acceleration of graphics as well (and can even use such acceleration by default when the game detects its availability).
- **Music:** Music is in the Standard MIDI File (SMF) format only. The General MIDI System level 1

per pixel (no more than 256 simultaneous colors) and all visuals in the game must be rendered in software (see the specification's definition of “rendered in software”). Further efforts to define 2-D game programming are in a page on a lean graphics API.² The 2-D limits in the specification apply, but instead of replacing a 2-D layer, the 3-D layer is simply a special sprite that covers the game screen (the usual size limits for sprites don't apply) and can have transparent and translucent pixels.³ Same as (2), but in addition, there are no tiles or 2-D layers (all the graphics are sprites). The tile-based limits in this specification also suit games that support only text display, and thus have visuals that resemble the text modes (as opposed to graphics modes) found in PCs and computer terminals. <https://peteroupc.github.io/graphicsapi.html>

¹³The Neo Geo (1990) has only one 2-D layer; the rest of the visuals are sprites drawn below that layer.

¹⁴Translucent pixels enable *alpha blending* techniques (the mixing of one image with another), also known as *compositing*. But alpha blending was “relatively new to PC games” at the time of *Quake*'s launch in 1996, according to Abrash (1997), and is practically not discussed at all in McCornack et al. (1995). Only images with opaque and/or transparent pixels tended to be supported in early-1990s video games. <https://ciechanow.ski/alpha-compositing/>

¹⁵This drawing can be done in many ways: for example, by designating pixels with a given label, say, 0, as transparent (in a process also known as *chroma keying*), or designating pixels that are the same as the upper-left pixel as transparent [Rodent 1994], or preparing a two-color mask identifying the pixels to be drawn [Gery 1992].

¹⁶https://registry.khronos.org/OpenGL/index_es.php

¹⁷https://registry.khronos.org/OpenGL/index_gl.php

should be followed for such files.¹⁸

The following are decided by the application:

1. The “**screen resolution**”: the width and height of the game screen image in pixels. The game screen image has no more than 307,200 total pixels (that is, no more pixels than a 640- $\times$ -480 screen, or a screen with 640 pixels horizontally and 480 vertically).¹⁹
2. If 3-D is supported:
 1. The maximum number of primitives shown simultaneously (“per frame”)²⁰. This number is no more than screen width times screen height divided by 24.
 2. Whether convex quadrilaterals (in addition to triangles and line segments) are 3-D primitives for purposes of the primitive limits.
 3. Which 3-D features are supported, as given in point 7 of the 3-D graphics section (for example, Gouraud shading, perspective correction, line drawing). (This can also be expressed as a subset of OpenGL 1.1 or OpenGL ES 1.1 that the application implements.)
 4. The maximum width and height of a texture in pixels. Each is no more than 256 and no more than the larger of the screen width and the screen height.
 5. Whether translucent pixels, transparent pixels, or both are supported in vertices and textures.
3. If 2-D tiles and sprites are supported:
 1. The number of 2-D layers (up to four).
 2. The number of bits per pixel that tiles have: 1, 2, 4, or 8.
 3. The width and height of each tile, in pixels. The width must be 32 or less, and the height must be 32 or less.²¹
 4. The maximum width and maximum height of a sprite in pixels. Each is no more than the following: Take the larger of the screen width and screen height, then divide by 4, then round up to the

¹⁸Standard MIDI files should be played back using a cross-platform open-source software synthesizer (see section “Building a Public-Domain music synthesis library and instrument banks”), using either FM or wave-table synthesis; most modern PCs no longer come with hardware synthesizers. I note that it’s possible to write an FM software synthesizer supporting every MIDI instrument in less than a million bytes of code. Standard MIDI files organize MIDI commands into up to 16 *channels*, each occupied by at most one “instrument” at a time. Under MPC Level 1, the first ten channels were intended for high-end synthesizers (where the tenth is percussion); the thirteenth through sixteenth, for low-end ones (sixteenth is percussion), and the nonpercussion channels were arranged in decreasing order of importance. This convention was abandoned with the rise in support for the General MIDI System level 1 (see Microsoft Knowledge Base article Q141087, “DOCERR: MarkMIDI Utility Not Provided in Win32 SDK”, 1995): now all 16 channels are supported (with only the tenth for percussion) and need not be arranged by importance.

¹⁹If the game screen image uses two colors only (such as black and white), the game could choose to allow it to have up to 800,000 total pixels. For example, a 1024 $\times$ 768 display has 786,432 total pixels. However, two-color graphical display modes larger than 307,200 total pixels are probably rare among consumers. The modern game *Return of the Obra Dinn* employs a two-color 800 $\times$ 450 display (378,000 total pixels) (but even so this resolution was “up from 640[$\times$]360”). In the Godot engine, the screen resolution corresponds to the “Viewport Width” (`window/size/viewport_width`) and “Viewport Height” (`window/size/viewport_height`) project settings, along with `window/stretch/mode=viewport` and `window/stretch/scale_mode=integer`. For the Unity engine, there is advice from 2019 relating to the visual style in “**8-bit**” and “**16-bit**” game consoles. In Unreal Engine, the screen resolution apparently corresponds to `ResolutionSizeX` and `ResolutionSizeY`. But a lighter-weight graphics engine than Unity, Unreal, or even Godot would better suit the spirit of this specification. <https://blog.unity.com/technology/2d-pixel-perfect-how-to-set-up-your-unity-project-for-retro-8-bits-games> <https://blog.unity.com/technology/2d-pixel-perfect-how-to-set-up-your-unity-project-for-retro-16-bit-games>

²⁰There are many **definitions of triangles** per frame. One definition is the triangles processed each frame after their vertices are transformed but before they are clipped (corresponding to the `GL_PRIMITIVES_GENERATED` query in **OpenGL 3.1**, section 2.16). Another definition (often called *visible triangles*) is the number of triangles under the first definition that are neither back-facing nor totally outside the view volume. Clipping these visible triangles to the view volume (which can sometimes produce more triangles) leads to a third definition: the triangles inside the volume that result after this clipping. A similar situation occurs with “primitives per frame” rather than “triangles per frame”. For purposes of this specification, an overestimate of primitives per frame (such as in the first definition) is better than an underestimate. The maximum number of primitives per frame in this specification was determined based on a target rate of 60 frames per second. <https://developer.arm.com/community/arm-community-blogs/b/mobile-graphics-and-gaming-blog/posts/triangles-per-second-performance-metric-or-chocolate-teapot> <https://registry.khronos.org/OpenGL/specs/gl/glspec31.pdf>

²¹The suggested width and height for tiles is 8 pixels by 8 pixels.

- nearest power of 2.
5. The maximum number of sprites displayed at a time on the game screen, no more than (screen width $\times$ screen height) / 256, rounded up.²²
 6. Whether sprites made of tiles, sprites made of pixels, or both are supported.

The following are optional features; the application decides:

1. Whether the application supports game screen resolutions with more pixels than a 640- $\times$ -480 screen, in addition to the primary resolution.
2. Whether hardware acceleration is supported in addition to rendering in software.
3. Whether tiles can be flipped horizontally and/or vertically.
4. Whether sprites can undergo a 2-D affine transformation (such as a rotation, enlargement, or shrinking).
5. Whether sprites can be flipped horizontally and/or vertically (even if affine transformation of sprites isn't supported in general).²³
6. Whether at least one of the 2-D layers can undergo a 2-D affine transformation.

Examples:

1. The following limits apply for various game screen resolutions. An application can decide on lower limits than the ones shown.

Resolution	Max. primitives per frame	Max. vertices per frame	Max. texture size	Max. sprite size	Max. sprites
640 $\times$ 480	12800	38400	256 $\times$ 256	256 $\times$ 256	1200
256 $\times$ 192	2048	6144	256 $\times$ 256	64 $\times$ 64	192
320 $\times$ 200	2666	8000	256 $\times$ 256	128 $\times$ 128	250
320 $\times$ 240	3200	9600	256 $\times$ 256	128 $\times$ 128	300
512 $\times$ 384	8192	24576	256 $\times$ 256	128 $\times$ 128	768
480 $\times$ 360	7200	21600	256 $\times$ 256	128 $\times$ 128	675
640 $\times$ 360	9600	28800	256 $\times$ 256	256 $\times$ 256	900
640 $\times$ 400	10666	32000	256 $\times$ 256	256 $\times$ 256	1000

2. For the screen resolution 640 $\times$ 480 and a frame rate of 60 frames per second, the maximum number of primitives per second is 768,000 ($= 60 \times (640 \times 480) / 24$); for 320 $\times$ 240, it is 192,000.
3. The following is one way to implement the 3-D portion of this specification:
 - Game screen resolution: 512 $\times$ 384.
 - Primitives per frame: up to 5000. Primitives are triangles and line segments, not quadrilaterals.

²²Tile- and sprite-based graphics were in place largely because they saved memory. Indeed, this system, present in the Nintendo DS and many earlier game consoles, was abandoned in the Nintendo 3DS in favor of a frame buffer (see the “Architecture of Consoles” series by Rodrigo Copetti). Game consoles employing tile-based visuals tended to limit not only the number of sprites per frame, but also the number of sprites per row of pixels, but a per-row limit is not adopted here. As for the per-frame limit, the Famicom/Nintendo Entertainment System (1983), SEGA Master System/SEGA Mark III (1985), and PC Engine/TurboGrafx 16 (1987) had a limit of 64 sprites; the Game Boy (1989), 40; the SEGA Mega Drive/Genesis (1988), 80; the Super Famicom/Super Nintendo Entertainment System (1990), 128; and the relatively expensive Neo Geo (1990), 381. Again, see the “Architecture of Consoles” series by Rodrigo Copetti.

²³SEGA arcade machines from the 1980s could scale (stretch or shrink) sprites horizontally and vertically in hardware (for example, *Turbo* [1981], *Hang-On* [1985], *Out Run* [1986]). In the Super Famicom/Super Nintendo Entertainment System (1990), sprites could not be scaled, but they could be flipped.

- Maximum texture size: 128×128 .
 - Implements a subset of OpenGL ES 1.1, rendered in software.
 - Has a game engine that renders only in that subset of OpenGL ES 1.1.
 - No translucent or transparent pixels.
4. One way to implement the 2-D limits is: Screen resolution 640×480 ; four 2-D layers; tiles have 4 bits per pixel; tiles are 16×16 pixels; maximum sprite size 64×64 pixels; up to 192 sprites at a time; sprites are made of pixels.

Note: For years earlier than 1999, some of the 3-D features mentioned in the specification (such as texture blending) might not be typical.

2.2 Classic Graphics in Scope

This specification for “classic graphics”²⁴ in modern games largely reflects the visual limitations of—

- consumer PCs (personal computers) released in the mid- to late 1990s,
- home computers released before 1995,²⁵
- game consoles (handheld and for TVs) released before 2000,
- arcade machines with similar performance to machines described earlier,²⁶
- personal digital assistants, graphical calculators, learning toys with an electronic display, and cellular phones with similar performance to consumer PCs from before 2000,²⁷
- videotex protocols before 1995²⁸, and
- the Game Boy Advance and Nintendo DS, both of which were released after 2000 but have relatively meager graphical ability.

In general, PC applications that feature classic graphics include:

1. Windows applications written for DirectX versions earlier than 7 and using Direct3D or DirectDraw for rendering visuals.
2. Windows games using GDI or **WinG**²⁹ for graphics and supporting Windows 98 or earlier. Examples are *Chip’s Challenge* for Windows (1992) and Brian Goble’s *The Adventures of MicroMan* (1993).
3. Games for MS-DOS or PC-9801 that were published before 2000. Examples are *Quake* (1996), *WarCraft* (1994), and the first titles of the Touhou Project series (1997-1998).
4. Games using **OpenGL**³⁰ 1.2 or earlier for graphics.

²⁴Saettler (1992) contains a rich discussion of graphics used in computer games and other audiovisual computer applications up to 1992. Not mentioned in that document are graphics resembling: (1) Segmented liquid-crystal displays, of the kind found in Nintendo’s Game & Watch and many Tiger Electronics handheld games. These are simple to emulate, though: design a screen-size image that assigns each segment a unique color and, each frame, draw black where the segments that are “on” are, and draw white (or another background) elsewhere on the screen. (2) Vacuum fluorescent displays, notable in user interfaces of some media player applications that resemble a “stereo rack system”.

²⁵Home computers include IBM PC compatibles, PC-88 and PC-98 families, ZX-Spectrum, Atari ST family, MSX family, Commodore 64, Amiga family, Commodore PET and VIC-20, BBC Micro, Amstrad CPC, Acorn Archimedes, Tandy TRS-80, Sharp X68000, Apple II, and computers running MS-DOS, Windows (up to Windows 98), Macintosh operating system (Mac OS) up to 9.x, NeXTSTEP, OS/2, or X Window System.

²⁶The first arcade video game, *Computer Space*, launched in 1971.

²⁷Examples are: (1) The **Sharp MI-Zaurus** (2000). (2) Cellular and feature phones that came with Java Micro Edition, its Mobile Information Device Profile (MIDP, **Java specification request 37** and **JSR 118**), and extensions (especially **JSR 184, Mobile 3D Graphics API**). (3) Feature phones and smartphones that supported the **OpenGL ES 1.1** or 1.0 specification (either the Common profile or the Common-Lite profile with fixed-point but not floating-point arithmetic). If there is interest, this exception could expand to years before 2009 and thus include certain video games for the original iPhone from 2007. https://dench.flatlib.jp/app/chiraks_em <https://jcp.org/en/jsr/detail?id=37> <https://jcp.org/en/jsr/detail?id=118> <https://jcp.org/en/jsr/detail?id=184> https://registry.khronos.org/OpenGL/index_es.php

²⁸Videotex is a communication system involving video screens. Some videotex protocols supported exchanging images and line drawings in addition to text, and it’s these that are of particular interest. See the videotex references in “Further Reading”. However, distribution of video games over videotex seems to be rare; an early example is *Dire Straits* (1982).

²⁹https://www.pcgamingwiki.com/wiki/List_of_WinG_games

³⁰https://registry.khronos.org/OpenGL/index_gl.php

5. So-called “multimedia titles” from the 1990s, or applications resembling interactive versions of books (generally reference and other nonfiction works), complete with sound, animation, and video. See “Multimedia PC Level 1 and Level 2 Specifications” by the Multimedia PC Marketing Council.

Video games released in 2000 or later are out of scope, even if they were designed for hardware that launched before 2000. The exceptions are:

- Video games released before 2010 for the Nintendo DS or the Game Boy Advance.
- Video games released before 2007 for personal digital assistants, graphical calculators, learning toys, and cellular phones that do not exceed the performance of consumer PCs released before 2000.³¹

NVIDIA GeForce 256 (late 1999) is out of scope. Some arcade machines, such as SEGA Model 3 (1996) and SEGA NAOMI (1998), may have 3-D rendering capabilities beyond what is “classic” here. Video games released in 1998 or 1999 for Dreamcast (1998) are borderline cases; they are at most barely within the spirit of this challenge.³²

One of the following games can be considered an upper limit to what is considered “classic graphics” in this specification.

- *Quake III Arena* (December 1999), which **required DirectX 7 and at least 64 million bytes of memory**³³.
- *Falcon 4.0* (1998).

2.3 Optional Limits

A game may impose further constraints to this specification (for example, to reduce the maximum number of 3-D triangles, to disallow 3-D graphics, to reduce the number of colors per tile allowed, or to **reduce to a limited set the colors**³⁴ ultimately displayed on screen). I would be interested in knowing about these limitations that a new game that adopts this document decides to impose.

Examples of optional constraints are the following:

- The game displays no more than 16 colors at a time.³⁵
- The game is limited to the 16 colors of the so-called *VGA palette*.
 - In the 8-bit-per-component color format, this palette’s colors are: light gray, that is, (192, 192, 192); or each color component is 0 or 255; or each color component is 0 or 128.
 - In the vertex color format, the closest colors to this palette are: 24/24/24; or each color component is 0 or 16; or each color component is 0 or 31.
- The game displays no more than 256 colors at a time.³⁶
- The game is limited to a fixed set of 256 colors, of which one is black and another is white.
- Drawing graphics with translucency (semitransparency) is not supported.
- All game files can be packaged in a ZIP file or Win32 program file that takes no more than—

³¹Examples are:(1) The **Sharp MI-Zaurus** (2000).(2) Cellular and feature phones that came with Java Micro Edition, its Mobile Information Device Profile (MIDP, **Java specification request 37** and **JSR 118**), and extensions (especially **JSR 184, Mobile 3D Graphics API**).(3) Feature phones and smartphones that supported the **OpenGL ES** 1.1 or 1.0 specification (either the Common profile or the Common-Lite profile with fixed-point but not floating-point arithmetic).If there is interest, this exception could expand to years before 2009 and thus include certain video games for the original iPhone from 2007. https://dench.flatlib.jp/app/chiraks_em <https://jcp.org/en/jsr/detail?id=37> <https://jcp.org/en/jsr/detail?id=118> <https://jcp.org/en/jsr/detail?id=184> https://registry.khronos.org/OpenGL/index_es.php

³²The following video games before 2000 are suspected, but not yet confirmed, to show more than 20,000 triangles per frame, a performance level beyond the spirit of this challenge: *The Lost World: Jurassic Park*, *Emergency Call Ambulance*, and *L.A. Machine Guns* for SEGA Model 3, and *Soul Calibur* and *Dead or Alive 2* for Dreamcast. See “**Seeking Comments**”.

³³https://www.dosdays.co.uk/topics/early_3d_games.php

³⁴https://peteroupc.github.io/classic-wallpaper#Color_Palettes

³⁵An example is *Loom* (1990).

³⁶This was recommended for Macintosh games in McCornack et al. (1995), notably because 8-bit-per-pixel images transfer faster and save memory over images with more bits per pixel; see also chapter 70 of Abrash (1997) (dealing with the porting of an 8-bit-per-pixel game to a 16-bit-per-pixel video card).

- 1,457,664 bytes (the capacity of a **file-allocation-table (FAT) formatted high-density 3.5-inch floppy disk**³⁷), or
- 1,213,952 bytes (the capacity of a FAT formatted high-density 5.25-inch floppy disk), or
- 730,112 bytes (the capacity of a FAT formatted normal-density 3.5-inch floppy disk), or
- 362,496 bytes (the capacity of a FAT formatted double-density 5.25-inch floppy disk), or
- 65,536 bytes,³⁸ or
- 681 million bytes (slightly less than the maximum total size of files in a formatted CD-ROM).
- The game uses no more than 16 million bytes of system memory at a time.
- The game uses no more than 655,360 bytes of system memory (plus 262,144 bytes of additional memory for graphical use only) at a time.³⁹
- The game is a Win32 application compatible with Windows XP.
- The game is a Win32 application compatible with Windows 98.
- The game aims for a rate of 30 frames per second.
- The game’s graphics must be *rendered in software* (see specification).
- The game’s rendering of visuals employs only 32-bit and smaller integers, integer arithmetic, and fixed-point arithmetic.⁴⁰
- The game’s rendering of visuals employs only 32-bit and smaller integers, integer arithmetic, fixed-point arithmetic, and arithmetic involving 32-bit floating-point numbers (IEEE 754 binary32, or the commonly implemented meaning of the C language’s `float` type).⁴¹
- The game renders only white line segments on a black background (or vice versa), and displays no more than 600 of those segments at a time. These line segments are drawn with an approximation at integer coordinates.
- The game runs with acceptable performance on **80486-compatible processors**⁴² at 33 or 50 megahertz.⁴³
- The game runs with acceptable performance on x86 processors at 100 megahertz.

2.4 Notes on Specification

This section has notes on this specification, such as how its requirements correspond to the graphics abilities of pre-2000 video games.

2.4.1 Screen resolutions

- Screen resolutions with more pixels than a 640- $\times$ -480 screen (such as 800 $\times$ 600) are not within the spirit of this challenge, even though more demanding games in the late 1990s, as well as the *PC 98 System Design Guide* (1997), aimed for the 800 $\times$ 600 resolution or higher for 3-D graphics. Indeed, for the most part, major game consoles and arcade machines in the 1990s and earlier supported resolutions with no more pixels than a 640- $\times$ -480 screen.⁴⁴

³⁷<https://fitsonafloppy.com/>

³⁸Popular file size limit of so-called “64K intros”.

³⁹MS-DOS applications are normally limited to 640 kibibytes or less of *conventional memory*, along with whatever memory is carried by the video card. (PCs running on the very early Intel 8086 and 8088 processors can map out no more than 640 kibibytes of system memory.) 262,144 bytes is the usual minimum of graphics memory for VGA video cards.

⁴⁰It wasn’t until the Pentium processor’s advent that floating-point arithmetic was embraced in 3-D game programming: for example, see chapter 63 of Abrash (1997).

⁴¹The use of 64-bit floating-point numbers (IEEE 754 binary64, or the commonly implemented meaning of the C language’s `double` type) is perhaps unusual in graphics programming before 2000.

⁴²<https://www.dosdays.co.uk/topics/cpus.php#i486>

⁴³This is the performance level targeted for the “**CyberVGA**” engine by Burak Yazar. <https://expfunction.itch.io/cybervga>

⁴⁴Moreover, PC games before 2000 that required screen resolutions larger than 640 $\times$ 480 are rare, and according to PCGamingWiki they include the following games (most of which are 2-D): *Timon & Pumbaa’s Jungle Games* (1995); *Tequila & Boom Boom* (1995); *Romance of the Three Kingdoms IV: Wall of Fire* (1995/1996); *Joint Strike Fighter* (1997), but only when run with the Glide programming interface; *Links LS: 1998 Edition* (1997); *Emergency: Fighters for Life* (1998); *Championship Manager: Season 99/00* (1999); *Heroes of Might and Magic III* (1999); *Alien Nations* (1999); *Pizza Syndicate/Fast Food Tycoon* (1999); *Age of Empires II: The Age of Kings* (1999).

- Screen resolutions that have been used in classic games include:⁴⁵
 - Video graphics array (VGA) display modes: 640×480 ,⁴⁶ 320×240 ,⁴⁷ 320×200 .⁴⁸
 - 4:3 aspect ratio: 640×480 ,⁴⁹ 512×384 ,⁵⁰ 400×300 ,⁵¹ 320×240 ,⁵² 256×192 ,⁵³ 160×120 .⁵⁴
 - Game console aspect ratios: 320×224 ,⁵⁵ 256×224 ,⁵⁶ 256×240 ,⁵⁷ 240×160 ,⁵⁸ 160×144 .⁵⁹
 - 5:4 aspect ratio:⁶⁰ 320×256 ,⁶¹
 - Two-color graphics: 720×348 ,⁶² 640×200 ,⁶³ 512×342 .⁶⁴
 - Enhanced Graphics Adapter aspect ratio: 640×350 .⁶⁵
 - 8:5 aspect ratio: 640×400 ,⁶⁶ 320×200 .⁶⁷
 - Other: 280×192 ,⁶⁸ 480×272 ,⁶⁹ 512×424 ,⁷⁰ 400×240 ,⁷¹ 384×224 ,⁷² 160×200 ,⁷³ 480×240 .⁷⁴

This is not a complete list. Some game consoles, such as the Nintendo 64, allowed games to alter the screen resolution during gameplay. **Arcade game graphics** are described later.

- As of early 1997, “[s]urveys indicate[d] that the great majority of [PC] users operate[d] in $640[\times]480$

⁴⁵In addition to the resolutions shown here, there are modern games that employ low resolutions with the same 16:9 aspect ratio as high-definition displays. These include 640×360 (*Blasphemous* [2019]); 400×225 (*Unsung*); 480×270 (*Enter the Gungeon*); 320×180 (*Celeste*). Information about game consoles’ screen resolution can be found in Rodrigo Copetti’s “Architecture of Consoles” series.

⁴⁶VGA mode 12h (16 colors); VGA mode 11h (2 colors).

⁴⁷Nintendo 3DS lower screen; larger VGA “mode X” (256 colors); most common Nintendo 64 resolution; SEGA Mega Drive PAL.

⁴⁸Commodore 64; NEC PC-8001; VGA mode 13h (256 colors), especially seen in MS-DOS games; Color/Graphics Monitor Adapter (CGA) 4-color mode; Atari ST 16-color mode; **Amiga NTSC**. <https://blog.johnnovak.net/2022/04/15/achieving-period-correct-graphics-in-personal-computer-emulators-part-1-the-amiga>

⁴⁹VGA mode 12h (16 colors); VGA mode 11h (2 colors).

⁵⁰One commonly supported “super-VGA” mode, especially in mid-1990s gaming, and which was also recommended by the *PC 98 System Design Guide* (chapter 14, item 20).

⁵¹One low resolution recommended by the *PC 98 System Design Guide* (chapter 14, item 20).

⁵²Nintendo 3DS lower screen; larger VGA “mode X” (256 colors); most common Nintendo 64 resolution; SEGA Mega Drive PAL.

⁵³Nintendo DS; NEC PC-6001; SEGA Master System/SEGA Mark III; MSX; Colecovision.

⁵⁴Rarely used VGA display mode.

⁵⁵SEGA Mega Drive/SEGA Genesis NTSC; Neo Geo NTSC; effective resolution for Sega Saturn on NTSC-compatible TVs. See also the note that follows.

⁵⁶Super Famicom/Super Nintendo Entertainment System NTSC; minimum resolution of PC Engine/TurboGrafx 16. According to Copetti’s “Architecture of Consoles” series, the vertical resolution is 224 rather than 240 because the upper and lower portions of the image were hidden behind the bezel of most NTSC-compatible TVs. As a result this is also the effective Famicom/Nintendo Entertainment System resolution for NTSC-compatible TVs.

⁵⁷Famicom/Nintendo Entertainment System (but see a previous note for NTSC-compatible TVs); Super Nintendo Entertainment System PAL.

⁵⁸Game Boy Advance.

⁵⁹Game Boy, Game Boy Color, SEGA Game Gear.

⁶⁰Aspect ratio found above all in PAL-compatible displays.

⁶¹Amiga PAL (same pixel spacing horizontally as vertically); Neo Geo PAL.

⁶²Hercules Graphics Card two-color.

⁶³Color/Graphics Adapter (CGA) two-color; NEC PC-8801 8-color mode; Atari ST 4-color mode.

⁶⁴12-inch classic Macintosh.

⁶⁵16 colors. See: IBM Corp., *IBM Enhanced Graphics Adapter*, Aug. 2, 1984.

⁶⁶NEC PC-9801 8-color mode; Atari ST two-color.

⁶⁷Commodore 64; NEC PC-8001; VGA mode 13h (256 colors), especially seen in MS-DOS games; Color/Graphics Monitor Adapter (CGA) 4-color mode; Atari ST 16-color mode; **Amiga NTSC**. <https://blog.johnnovak.net/2022/04/15/achieving-period-correct-graphics-in-personal-computer-emulators-part-1-the-amiga>

⁶⁸Apple II.

⁶⁹PlayStation Portable.

⁷⁰MSX 2.

⁷¹Effective resolution of Nintendo 3DS upper screen without parallax effect.

⁷²Virtual Boy.

⁷³One “Tandy graphics adapter” mode.

⁷⁴Minimum resolution for “handheld PCs” (*Windows CE Programmer’s Guide*, MSDN Library, June 1998).

resolution with 256 colors”.⁷⁵

- A game can support—

- multiple sizes for the area of the screen where the game’s action is drawn, or
- pixel-column or -row doubling as a “quality” setting,

or both features, without changing the size of the game’s image. For example, the original *Doom* (1993) supported several sizes of this kind (on PC, they were 96×48 , 128×64 , 160×80 , and so on up to 288×144 , as well as 320×168 and 320×200) and optional pixel-column doubling.⁷⁶

- Games within the scope of this challenge are meant to be run in a desktop window if the player’s display is 800×600 pixels or larger. The same is true if the game’s resolution is 620×420 or smaller and the player’s display is 640×480 . The game may also support full-screen display.
- For games that simulate the resolution of video game consoles supporting NTSC- or PAL-compatible TV screens,⁷⁷ a vertical resolution of—
 - 240 or 288 rows of video (respectively) in *progressive scan* at nearly 60 frames or 50 frames per second, or
 - 480 or 576 rows of *interlaced video* (in two passes, each pass drawing every other row) at nearly 30 frames or 25 frames per second,

is common. (By contrast, the horizontal resolution could vary in such cases.) Such TV screens draw 240 or 288 visible rows from top to bottom, whether in interlacing or progressive scan. In the case of some NTSC-compatible TVs, though, the upper and lower portions of the image are hidden by the bezel, so that, in effect, only 224 rows in progressive scan or 448 rows of interlaced video may be visible (see Rodrigo Copetti’s “Architecture of Consoles” series).

2.4.2 Frame rate

- No particular frame rate is required.⁷⁸
- Possible choices for the frame rate include 20, 30, 40, and 60 frames per second. Rates beyond 60 were rare before 2000.⁷⁹
- For NTSC- and PAL-compatible TV screens, see the remark on frames per second in the previous section.
- *Doom* (1993) operated at 35 frames per second but could not be run at that rate (under default settings) by typical PCs of the time.⁸⁰
- For comfort reasons, a minimum frame rate may be required for video games that offer “**3-D vision**”⁸¹ by rendering multiple views of the scene at a time, in conjunction with special glasses (for example, a SEGA Master System accessory) or a virtual-reality headset (for example, Nintendo’s Virtual Boy). But such games were rare before 2000.

⁷⁵S. Pruitt, “Frequently Asked Questions About HTML Coding for Internet Explorer 3.0”, updated Jan. 30, 1997.

⁷⁶Fabien Sanglard, *Game Engine Black Book: Doom*. <https://fabiansanglard.net/gebb/>

⁷⁷“NTSC” stands for the National Television Standards Committee of the Electronics Industries Association. “NTSC” often refers to the video display protocol known as RS-1070A. “PAL” stands for phase alternating line. For information on NTSC- and PAL-compatible video, see *Conventional Television Systems* (listed in the “Further Reading” section of this article).

⁷⁸Until the early 1990s, the number of color samples (pixels) an application can transfer per second was usually small, limiting the supported size and frame rate for arbitrary video content. Indeed, for example, MPC Level 1 recommended that video cards be able to transfer up to 8-bit-per-sample graphics at a rate of 140,000 samples per second or faster given 40 percent of CPU bandwidth. MPC Level 2 upped this recommendation to 1.2 million samples per second (sufficient for 320×240 video at 15 frames per second, the recommendation in Microsoft Knowledge Base article Q139826, “AVI Video Authoring Tips & Compression Options Dialog Box”, 1995). Both recommendations are far from the 6.144 million samples per second needed to display 640×480 video smoothly at 20 frames per second.

⁷⁹Most of the VGA standard’s display modes, including the 320×200 mode, ran at 70 hertz, but the 320×240 and 640×480 modes ran at 60 hertz instead. However, even video games with VGA video tended not to target 70 frames per second.

⁸⁰Fabien Sanglard, *Game Engine Black Book: Doom*. <https://fabiansanglard.net/gebb/>

⁸¹https://www.pcgamingwiki.com/wiki/Glossary:Native_3D

2.4.3 3-D graphics

- The *PC 99 System Design Guide* sections 14.27 to 14.34 gives guidelines on 3-D graphics support for PCs to be launched in 1999.⁸² This challenge recommends the writing of game software with graphics performance as good as hardware meeting such guidelines, except for the screen resolution, frame rate, and double buffering requirements.
- An application may choose to support stencil buffers, bump mapping, and three- or four-texture blending, but these are borderline pre-2000 graphics capabilities.⁸³
- This specification allows for:
 - Prerendered graphics, to simulate showing highly detailed imagery. Examples of video games with prerendered graphics are *Space Quest V* (1993), *Myst* (1993), *Star Wars: Rebel Assault* (1993), *Resident Evil* (1996), the original *Final Fantasy VII* on PlayStation (1997), and the arcade games *GP World* (1984) and *Dragon's Lair* (1983).
 - Drawing a 3-D graphic as a *voxel mesh*⁸⁴ (formed from point samples in 3-D, rather than 2-D, called *voxels*)⁸⁵, as long as the triangle limits are respected.
- The following are not within the spirit of this challenge:
 - Displaying more than 20,000 triangles at a time (per frame), even for higher screen resolutions. Most 3-D video games before 2000 displayed well fewer than that, but there may be exceptions.⁸⁶ Moreover, a rate of much more than 20,000 triangles per frame is poorly suited for rendering in software.
 - Phong shading (per-pixel specular highlighting), ray tracing, and path tracing, which were too slow for real-time use in the 20th century. Exceptions include:
 1. Per-screen-column *ray casting*, of the kind popularized in *Wolfenstein 3-D* (1992).⁸⁷
 2. *Light maps* with precomputed lighting (for example, Abrash 2007, chapters 68 and 69).
- It wasn't until 1995 that 3-D video cards became widely available for consumer PCs.⁸⁸ In 3-D video games for PCs “[i]n 1995/1996, it was not uncommon to have 30-50% of the game screen filled with polygons without textures” (according to an **article**⁸⁹ that compared *Havoc* [1995] with *Mortal Kombat 4* [1997]).
- This specification is not centered on video games that offer “3-D vision” (see note under “Frame rate”), given how rare they were before 2000.
- The following is an alternative primitive drawing rate an application can use. Nadeau, D.R., “Optimizing the Performance of VRML Worlds”, *Dr. Dobbs' Journal*, July 1996, mentions: “[A] typical software implementation of the graphics pipeline for a Pentium PC can draw about 90,000 triangles per second. At a 60-Hz video refresh rate, the software can draw about 1500 triangles per refresh”. Assuming the Pentium processor's speed is 100 megahertz this suggests a target rate of 15 triangles per megahertz per frame. For example, if the target processor speed is 250 megahertz or faster, the target rate is 3750 triangles per frame.

⁸²The *PC 98 System Design Guide*, chapter 14, items 24 to 30, have largely the same guidelines. The *PC 97 Hardware Design Guide*, chapter 15, items 24-28 and 30-31, have weaker guidelines.

⁸³So is environment mapping, but this feature is simply the generating of texture coordinates in software for a suitably crafted texture image. See Woo et al. (1997), chap. 9, section “Environment Mapping”.

⁸⁴<https://blog.danielschroeder.me/blog/voxel-renderer-objects-and-animation>

⁸⁵Examples include Amanatides and Woo (1987), chapter 16 of Lamothe (1995), chapter 7 of Anderson (1995), and the **voxel space** algorithm for terrain rendering in U.S. Patent 6020893A, now expired, and used in *Comanche: Maximum Overkill* (1992). <https://github.com/s-macke/VoxelSpace>

⁸⁶The following video games before 2000 are suspected, but not yet confirmed, to show more than 20,000 triangles per frame, a performance level beyond the spirit of this challenge: *The Lost World: Jurassic Park*, *Emergency Call Ambulance*, and *L.A. Machine Guns* for SEGA Model 3, and *Soul Calibur* and *Dead or Alive 2* for Dreamcast. See “**Seeking Comments**”.

⁸⁷Fabien Sanglard, *Game Engine Black Book: Wolfenstein 3D*; Lampton, C., *Gardens of Imagination: Programming 3D Maze Games in C/C++*, Waite Group Press, 1994, ISBN 1-878739-59-X. <https://fabiansanglard.net/gebb/>

⁸⁸By contrast, 3-D video cards have been offered for professional-use computers since the mid-1980s; the first such cards for PCs that supported real-time display were **introduced in 1988**. <https://retro.swarm.cz/sgi-irisvision-add-in-3d-accelerator-for-pc-1990/>

⁸⁹<https://retro.swarm.cz/s3-virge-325-vx-dx-gx-gx2-series-of-early-3d-accelerators-deep-dive/>

2.4.4 Graphics in video arcade games

- From 1971 to 1979, arcade games tended to have white visuals drawn on a black screen; the screen image was one bit per pixel. Some arcade cabinets in the late 1970s had a colored semitransparent overlay in front of the screen to lend color to the otherwise white graphics (an example is *Lunar Rescue* [1979]).
- The practice of drawing graphics on a mostly black screen continued up to the early 1980s, but this time they were colored by the game itself rather than by an overlay.
- The visuals of some arcade games from 1979 to the early 1980s were made of white line segments. Notable examples are *Asteroids* (1979) and *Battlezone* (1980). In *Space Fury* (1981) and *Star Wars* (1983), these line segments were even multicolored.
- Graphics made of tiles and sprites were popularized by the arcade game *Galaxian* (1979).
- Most arcade games before 1995 had a resolution from 46,080 pixels (192×240 or 240×192) through 98,304 pixels (256×384 or 384×256). The following were exceptions:
 - Certain two- or three-screen games by Taito (for example, *Darius* [1987] and *The Ninja Warriors* [1987]).
 - The Midway MCR family of arcade machines (1980 to about 1986) had a screen resolution of 480×480 , 512×448 , or 512×480 , but backgrounds were drawn at half the resolution of sprites. Examples include *Tapper* (1982) and *Wacko* (1983).
- If an arcade game employed 2-D visuals with tiles and sprites, each tile and sprite tended to use no more than 16 colors, and semitransparency was not used.
- There have been so-called cinematic arcade video games, such as *Wild Gunman* (1974) and *Dragon's Lair* (1983).

2.4.5 Screen image effects (filters)

Outside the scope of this challenge are effects that—

- modify the game screen image to emulate CRT displays⁹⁰,
- **scale**⁹¹ the game screen to fit the height or width of the player's display,⁹² or
- simulate the interlaced video display of certain TV screens, especially NTSC- and PAL-compatible ones.

This specification assumes those effects are not in place. A game can have those effects if it wishes, but they should be in-game settings.

2.4.6 Sounds

- Besides the limitation on music, this specification has no further limitations on sounds.
- Early game consoles supported sound only through one or more *programmable sound generators*, such as square and triangle wave generators, as opposed to digitized sounds⁹³. Games that choose to constrain file size may wish to implement software versions of programmable sound generators for at least some of their sounds.

⁹⁰CRT displays, or cathode-ray-tube displays, were the typical kind of computer monitors and TVs in the 1980s and 1990s.

⁹¹<https://www.pcgamingwiki.com/wiki/Glossary:Scaling>

⁹²Effects to scale the game screen include so-called “pixel-art scaling algorithms” such as HQX and 2xSaI, as well as bilinear or point filtering. Effects to scale the game screen do not include the decoding of small videos to fit the *game screen*, as opposed to the player's display. It was common for 1990s games to have videos smaller than the game screen and to scale those videos to fit the game screen “on the fly”, in the process of displaying them. For example, such a game could decode videos of size 160x100 to fit a game screen of 320×200 . (See, for instance, Nigel Thompson, “**Stretching 256-Color Images Using Interpolation**”, Microsoft Developer Network, March 7, 1995.) [[https://learn.microsoft.com/en-us/previous-versions/ms969922\(v=msdn.10\)](https://learn.microsoft.com/en-us/previous-versions/ms969922(v=msdn.10))]([https://learn.microsoft.com/en-us/previous-versions/ms969922\(v=msdn.10\)](https://learn.microsoft.com/en-us/previous-versions/ms969922(v=msdn.10)))

⁹³Sound today is most commonly digitized by pulse-code modulation (PCM), and PCM-digitized sound is often stored in computer files ending in “.WAV”.

- When digitized sounds are supported in classic games, they typically have a sample rate of 8000, 11,025, 22,050, or 44,100 hertz, are either mono or stereo, and take 8 or 16 bits per sample.⁹⁴

2.4.7 Memory

- This specification does not impose a limit on graphical memory use (akin to the video memory, or VRAM, of a video card). Such a limit is optional. One suggested example, given in kibibytes, is the screen width times screen height divided by 24, which is slightly less than 13.2 million bytes for 640 × 480 resolution. (A kibibyte is 1024 bytes.) Imposing a limit on graphical memory use does not limit the size or number of textures, 3-D models, images, or other resource files a game can have.⁹⁵
- According to “**Typical PCs Each Year**”⁹⁶, the following ranges of system memory were typical for PCs sold in the specified years:⁹⁷
 - 1994: 4MB to 8 MB, with more expensive PCs having 16 MB.⁹⁸
 - 1997: 8MB to 32MB.⁹⁹
 - 1998: 32MB to 128MB.¹⁰⁰

2.5 Seeking Comments

As with the rest of this open-source article, **comments on this specification**¹⁰¹ are welcome. But most useful would be comments that improve or refine the specification to fit the graphical abilities of pre-2000 video games.

Examples are comments that give *measurements* (or references to other works that make such measurements) on the graphical performance actually achieved, in normal gameplay, by video games released in 1999 and earlier (or released in, say, 1994 and earlier) for home computers or game consoles. (It bears repeating: *measurements*, not inferences or guesses from screenshots or videos.)

This includes statements like the following, with references or measurements:

- “Game X shows up to Y polygons per frame at screen resolution W [and Z frames per second]”.
 - Statements like this that relate to polygons or triangles per frame are hard to find and often anecdotal, and they cannot always be inferred from screenshots or videos of gameplay. For example: (1) “A typical scene in a current [PC] application has 2000 to 2500 triangles per frame” (Fosner 1999)¹⁰². (2) According to the help for the 3DMark2000 benchmark, that benchmark comes with two game scenes that average up to 9,400 polygons in low detail and up to 55,000 in high detail. (3) The game engine for *SpecOps: Rangers Lead the Way* (1998) targeted 10,000 triangles per frame¹⁰³. So did *Quake III Arena* (1999) (John Carmack .plan, Sep. 2, 1999).
- “Scenes in game X have Y triangles on average”.
- “Game X shows no more than [16 or 256] simultaneous colors”.
- “Game X uses Y bytes of memory while running on Windows 98”.¹⁰⁴

⁹⁴The MPC Level 1 specification required support in “multimedia PCs” for playback of at least 8-bit-per-sample mono digitized sound at 11,025 and 22,050 hertz. MPC Level 2 required support in “multimedia PCs” for playing back at least 16-bit-per-sample stereo digitized sound at 44,100 hertz.

⁹⁵PC games released in 1999 tended to require 32 million bytes of system memory. Meanwhile, *Quake* (1996) required 8 million and recommended 16 million bytes of system memory.

⁹⁶https://www.dosdays.co.uk/topics/typical_pc_per_year.php

⁹⁷“MB” is ambiguous here; it often means either one million bytes or 1024 times 1024 bytes.

⁹⁸“Typical PCs in 1994”, <https://www.dosdays.co.uk/topics/1994.php>.

⁹⁹“Typical PCs in 1997”, <https://www.dosdays.co.uk/topics/1997.php>.

¹⁰⁰“Typical PCs in 1998”, <https://www.dosdays.co.uk/topics/1998.php>.

¹⁰¹https://www.reddit.com/r/retrogamedev/comments/1rl36fo/pre2000_computer_graphics_for_modern_video_games/

¹⁰²R. Fosner, “DirectX 6.0 Goes Ballistic With Multiple New Features And Much Faster Code”, *Microsoft Systems Journal* January 1999.

¹⁰³“**Postmortem: Zombie’s SpecOps: Rangers Lead the Way**”, Jan. 31, 2000. <https://www.gamedeveloper.com/design/postmortem-zombie-s-i-specops-rangers-lead-the-way-i>

¹⁰⁴The system requirements of games for home computers offer approximations to statements like this, but they are not

- “Game X shows up to Y sprites at a time [at screen resolution Z]” (for 2-D games such as those built using the tool Director, then by Macromedia).
- The 2-D game X, from year Y, supports a **given 2-D graphics capability**¹⁰⁵ (for example, 2-D rotations of sprites; filling ellipses with a solid color; flood filling; antialiasing of lines and shapes; translucent alpha blending; translucent sprites).
- The 3-D game X, from year Y, supports a **given 3-D graphics capability**¹⁰⁶ (for example, Gouraud shading, bump mapping, edge antialiasing, alpha blending, texture mapping of most polygons in a scene, or MIP mapping).

(Those statements will also help me define constraints for video games up to an earlier year than 1999.)

Statements like the following are also useful, with references:

- “In year X [1999 or earlier], Y% of PC users used screen resolution Z”.
- In year X, a given 3-D graphics capability became typical in 3-D video games.
- In year X, a given 2-D graphics capability became typical in 2-D video games.
- “In year X [1999 or earlier], Y% of home computers in use were equipped with Z million bytes of memory”.
- “In year X, Y% of home PCs were equipped with 3-D video cards”.
- A market-share-weighted average of system memory requirements of video games in year X.
- On a market-share-weighted basis, X% of video games in year Y—
 - ran on [16 or 256]-color display modes, or
 - had real-time 3-D rendering, or
 - were played on arcade machines, or
 - were played on PCs, or
 - were played on game console Z.

By contrast, statements like the following are not very useful, since they often don’t relate to the actual performance of specific video games:

- “Game console/video card X can process up to Y **triangles per second**¹⁰⁷”.
- “Game console/video card X can render up to Y polygons per frame”.
- “Game console/video card X can render up to Y pixels per second”.
- “Game X renders Y triangles per second”, without stating the frame rate or the screen resolution.
- “Game X issues Y draw calls per frame”, since a single draw call can draw one triangle or tens of thousands.
- “Character models in game X average Y triangles”.

The following are examples of the kind of statements desired:

- *Actua Soccer (VR Soccer '96)* (1995) **averaged 776 triangles per frame** at 640×480 resolution.
- *Terminal Velocity* (1995) **averaged 498 triangles per frame** at 640×480 resolution.
- A benchmark of *Quake III Arena* averaged about 3,250 and topped out at about 6,970 triangles per frame after back-face culling, at screen resolution 640×480 (Antochi et al. 2003)¹⁰⁸, (Antochi et al. 2004)¹⁰⁹.

perfect. Indeed, a game might use less memory than the operating system’s minimum requirements, such as Windows 98’s system memory requirement of 16 million bytes.

¹⁰⁵https://peteroupc.github.io/graphicsapi.html#2_D_Graphics

¹⁰⁶https://peteroupc.github.io/graphicsapi.html#3_D_Graphics

¹⁰⁷<https://developer.arm.com/community/arm-community-blogs/b/mobile-graphics-and-gaming-blog/posts/triangles-per-second-performance-metric-or-chocolate-teapot>

¹⁰⁸Antochi, Iosif, et al. “3D Graphics Benchmarks for Low-Power Architectures.” 14th Annual Workshop on Circuits, Systems and Signal Processing. 2003.

¹⁰⁹Antochi, Iosif, et al., “GaalBench: a 3D graphics benchmark suite for mobile phones”, *ACM SIGPLAN Notices* 39(7), 2004.

- The game *Star Wars Racer* (1999) **targeted a limit of 3000 polygons per frame**¹¹⁰ as well as texture sizes that appeared to respect the Nintendo 64’s 4096-byte texture cache.

2.6 Further Reading

- Abrash, M., *Michael Abrash’s Graphics Programming Black Book: Special Edition*¹¹¹, 1997.
- (Akenine-)Möller, T., Haines, E., *Real-Time Rendering* (first edition), 1999.
- Amanatides, John, and Andrew Woo. “A fast voxel traversal algorithm for ray tracing.” *Eurographics*. Vol. 87. No. 3. 1987. <http://www.cse.yorku.ca/~amana/research/grid.pdf>
- Anderson, G (ed.), *More Tricks of the Game Programming Gurus*, Sams, 1995. ISBN 0-672-30697-2.
- CSA T500-1983/ANSI X3.110-1983/FIPS PUB 121, “Videotex/Teletext Presentation Level Protocol Syntax”, also known as the North American Presentation Layer Protocol Syntax.
- Gery, R., “Bitmaps with Transparency”, Microsoft Developer Network, Jun. 1, 1992.
- Houston, B., “Rendering Real-Time 3D Before GPUs”, Jul. 17, 2026. <https://ben3d.ca/blog/rendering-real-time-3d-before-gpus>
- International Telecommunication Union, *Conventional Television Systems*, ITU-R BT.470-6, 1998.
- International Telecommunication Union, *International Interworking for Videotex Services*, ITU-T T.101, 1988.
- Lamothe, A., et al., *Tricks of the Game Programming Gurus*, Sams, 1994. ISBN 0-672-30846-0.
- Lamothe, A., *Black Art of 3D Game Programming*, Waite Group Press, 1995. ISBN 1-57169-004-2.
- Lamothe, A., *Tricks of the 3D Game Programming Gurus: Advanced 3D Graphics and Rasterization*, Sams, 2003. Published after 1999, but most of the 3-D capabilities discussed there are within the spirit of this specification. ISBN 0-672-31835-0.
- Lamothe, A., *Tricks of the Windows Game Programming Gurus*, Sams, 1999. ISBN 0-672-31361-8.
- J. McCornack et al., *Tricks of the Mac Game Programming Gurus*, Hayden Books, 1995. ISBN 1-56830-183-9.
- PC hardware guidelines:
 - Multimedia PC Marketing Council, “Multimedia PC Level 1 and Level 2 Specifications” (May 1993), also in Microsoft Knowledge Base article Q106055 (1993). The two specifications are called MPC Level 1 (from 1990) and MPC Level 2 (from 1993) in this document.
 - In *Dr. Dobbs’ Journal*, Sep. 1995 (“Of Interest”), it was announced that the Multimedia PC Software Publishers Association released an MPC Level 3 specification.
 - Microsoft Corp., *Hardware Design Guide for Microsoft Windows 95*, Microsoft Press, 1995. ISBN 1-55615-642-1.
 - Microsoft Corp., *PC 97 Hardware Design Guide*, Microsoft Press, 1997. ISBN 1-57231-381-1.
 - Intel & Microsoft, *PC 98 System Design Guide*, Microsoft Press, 1997. ISBN 1-57231-716-7.
 - Intel & Microsoft, *PC 99 System Design Guide*, Microsoft Press, 1998. ISBN 0-7356-0518-1.
- Roca, Jordi, et al., “**Workload Characterization of 3D Games**¹¹²”, *2006 IEEE International Symposium on Workload Characterization*. IEEE, 2006. Study on measuring certain features of 3-D games that are of interest in this specification, including triangles per frame. See the **attila-sim repository**¹¹³.
- Rodent, H., “Animation in Win32”, Microsoft Developer Network, Feb. 1, 1994.
- Saettler, Matt, “Graphics Design and Optimization”, Multimedia Technical Note (Microsoft), 1992.
- TeleGrafix Communications, Inc., “RIPscrip Graphics Protocol Specification”, revision 1.54, July 19th, 1993.
- Thompson, N., *3D Graphics Programming for Windows 95*, Microsoft Press, 1996. ISBN 1-57231-345-5.
- Thompson, N., *Animation Techniques in Win32*, Microsoft Press, 1995. ISBN 1-55615-669-3.
- Woo, Mason, et al., *OpenGL Programming Guide: The Official Guide to Learning OpenGL, Version*

¹¹⁰<https://x.com/jonk1969/status/1636183310203498496>

¹¹¹<https://github.com/jagregory/abrash-black-book>

¹¹²<https://ieeexplore.ieee.org/abstract/document/4086130>

¹¹³<https://github.com/attila-gpu/attila-sim>

1.1, second edition, Addison Wesley Longman, 1997. ISBN 0-201-46138-2.

3 Building a Public-Domain music synthesis library and instrument banks

To improve support for MIDI (Musical Instrument Digital Interface) music playback in open-source and other applications, I challenge the community to write the following items, all of which must be released to the public domain or under the Unlicense.

- A cross-platform open-source library for *software* synthesis (translation into digitized sound) of MIDI data stored in standard MIDI files (SMF, .mid), using instrument sound banks (synthesizer banks) in SoundFont 2 (.sf2), Downloadable Sounds (.dls), and in OPL2, OPL3, and other FM synthesis sound banks, and possibly also in Timidity++/UltraSound patch format (.cfg, .pat). (Similar to *Fluidsynth*, but in the public domain or under the Unlicense. Instrument sound banks are files that describe how to render MIDI instruments as sound. In addition, the source code in the nonpublic-domain *foo_midi*, *libADLMIDI*, *libOPNMIDI*, *OPL3BankEditor*, and *SpessaSynth* may be useful here, but review their licenses first.)
 - The library should support popular loop-point conventions found in MIDI files.
 - The library should support seeking of MIDI files such that a pause and resume function can be offered by a media player.
- An instrument sound bank for wave-table synthesis of all instruments and percussive (drum) noises in the General MIDI System level 1 specification.
 - Instruments should correspond as closely as possible to those in that specification, but should be small in file size or be algorithmically generated.
 - Instruments can be generated using the public-domain single-cycle wave forms found in the AdventureKid Wave Form collection, found at: **AKWF-FREE**¹¹⁴.
 - The samples for each instrument may, but need not, be generated by an algorithm, such as one that renders the instrument's tone in the frequency domain. An example of this is found in **com.sun.media.sound.EmergencySoundbank**¹¹⁵, which however is licensed under the GNU General Public License version 2 rather than public domain.
 - The instrument sound bank should be in either SoundFont 2 (.sf2) or Downloadable Sounds (.dls) format. See next section on a challenge to writing a guide on sound bank development.
 - The volume of all instruments in the sound bank should be normalized; some instruments should not sound louder than others.
- An instrument sound bank for FM synthesis of all instruments and percussive noises in the General MIDI System level 1 specification. Instruments should correspond as closely as possible to those in that specification.

4 Other Challenges and Projects

Other challenges and projects I make to the computer community.

None of the following creations should be generated by artificial-intelligence tools or with their help.

4.1 Classic desktop wallpaper

See the “**peteroupc/classic-wallpaper**¹¹⁶” repository for a challenge on creating tileable desktop wallpapers with a limited selection of colors and limited dimensions in pixels — such wallpapers are getting

¹¹⁴<https://github.com/KristofferKarlAxelEkstrand/AKWF-FREE>

¹¹⁵<https://github.com/apple/openjdk/blob/xcodejdk14-release/src/java.desktop/share/classes/com/sun/media/sound/EmergencySoundbank.java>

¹¹⁶<https://peteroupc.github.io/classic-wallpaper>

ever harder to find because desktop backgrounds today tend to cover the full computer screen, to employ thousands of colors, and to have a high-definition resolution (1920×1080 or larger).

4.2 Button and border styles for classic interfaces

See “**Traditional User-Interface Graphics**”¹¹⁷ for a challenge on writing computer code (released to the public domain or under the Unlicense) to draw button and border styles for classic graphical user interfaces.

4.3 Sound bank development guide

Write an open-source and detailed guide on using free-of-cost software to produce decent-quality instrument banks from the recordings of real musical instruments (rather than copying or converting other instrument banks or recording from commercial synthesizers). See the section on **building instrument banks, earlier**. For this purpose, a sound bank in SoundFont 2 or Downloadable Sounds format that is of decent quality is about 4 million bytes in size.

4.4 Guide for creating 3-D models in the pre-2000 style

Develop a guide for creating 3-D models for use in modern video games that follow the **specification** given earlier on classic (pre-2000) 3-D graphics, in a similar vein to “**Game-Ready 3D Models: Requirements, Creation, and Export**”¹¹⁸ ¹¹⁹. Notably, no shader-based techniques should be required for any such models, and advice should apply to models for a game just as though the game were developed in 1999 (or an earlier year) rather than today, but the use of modern creation tools is allowed. (For example, instead of normal, roughness, or ambient-occlusion maps, late-1990s 3-D game models typically employed light maps and bump maps, and such models were generally much coarser than today’s models.)

4.5 Book-form tutorial on pre-2000 computer graphics programming

Write a free and open-source tutorial on game graphics programming using the pre-2000 graphic style emphasized in the **specification**¹²⁰ given earlier, with the following features:

1. The tutorial is book-length, printable, and programming-language neutral.
2. The tutorial should cover all 2-D and 3-D concepts needed to exploit the specification fully, such as the concepts of tiles, sprites, 2-D layers, textures, 3-D triangles, and features for 3-D graphics. Coverage of graphics programming with a limited-color frame buffer (such as 16 colors or 256 colors) is recommended.
3. The tutorial explains math concepts to readers as necessary, without assuming prior knowledge of math higher than basic algebra.
4. The tutorial should include checks for understanding, guided practice, and independent practice exercises. The tutorial may also include review (retrieval practice) of past concepts.
5. The tutorial does not require the use of hardware acceleration, OpenGL, or Direct3D in the practice exercises. (See definition of “rendered in software” in the specification.)
6. As a strong recommendation, the guided and independent practice can lead to readers writing a feature-rich software renderer that implements the pre-2000 graphics specification without third-party software libraries.

Similar tutorials are *Computer Graphics from Scratch*¹²¹ by Gabriel Gambetta and the **tiny-renderer**

¹¹⁷https://peteroupc.github.io/classic-wallpaper/docs/uielements.html#Button_and_Border_Drawing_Challenge

¹¹⁸<https://threedium.io/3d-model/game-ready>

¹¹⁹The “Game-Ready 3D Models” guide was designed for high-system-resource games from 2024 or so, but appears to have been generated by artificial-intelligence tools, which are not allowed for this project.

¹²⁰https://peteroupc.github.io/graphics.html#Graphics_Challenge_for_Classic_Style_Games

¹²¹<https://gabrielgambetta.com/computer-graphics-from-scratch/>

tutorial¹²² by Dmitry Sokolov, although they are not always geared toward the visuals in pre-2000 video games. For example, Gambetta covers ray tracing (which is outside this specification’s scope) but not sprites or other 2-D concepts.

Example: A potential list of topics the tutorial can have:

- The game screen; Line drawing; Polygon and triangle filling; Circles and ellipses; Flood filling; Text rendering using line drawing; Shape drawing review;
- Colors and color palettes; Image copying; Image stretching; Sprites; Tile maps and layers; Sprite animation; Text rendering using sprites and tiles; Image and sprite review;
- Palette animation; Pixel formats; 2-D affine transformations; 2-D review;
- 3-D lines and triangles; 3-D transformations (including perspective correction); The “camera”;
- Per-column ray casting; Hidden surface removal; Triangle clipping; Z buffering;
- Shading (flat and Gouraud) and texture mapping; Texture blending; Specular highlighting;
- Fixed-point numbers; MIP mapping; Fog; Combining 2-D and 3-D; 3-D review.

Examples of exercises for line drawing are an exercise for drawing polygons, and another for plotting Bézier curves.

5 Acknowledgments

I acknowledge the advice of the `gameengine devs` and `programming` communities on Reddit.

6 License

Any copyright to this page is released to the Public Domain. In case this is not possible, this page is also licensed under **Creative Commons Zero**¹²³.

7 Notes

¹²²<https://haqr.eu/tinyrenderer/>

¹²³<https://creativecommons.org/publicdomain/zero/1.0/>